Applied AI / ML Engineering

Take-Home Assignment — Submission Template

Replace every [bracketed placeholder] and every dashed box with your own content. Do not delete or rename the section headers — graders read against them. Keep prose tight; we score evidence, not adjectives.

Candidate details

Field	Your entry
Full name	Akash Deep
Email	akashgsdeep12@gmail.com
Date submitted	2026-10-23
Repository link	https://github.com/akashdeep4303/syllabus/tree/main
Total time spent	~8 h
Problem 1 — vector store chosen	ChromaDB (embedded, cosine HNSW)
Problem 2 — judge model / generator model	judge: llama3.2:3b · generator: llama3.2:3b (same family; independent env keys — see 2.4)

How to use this template

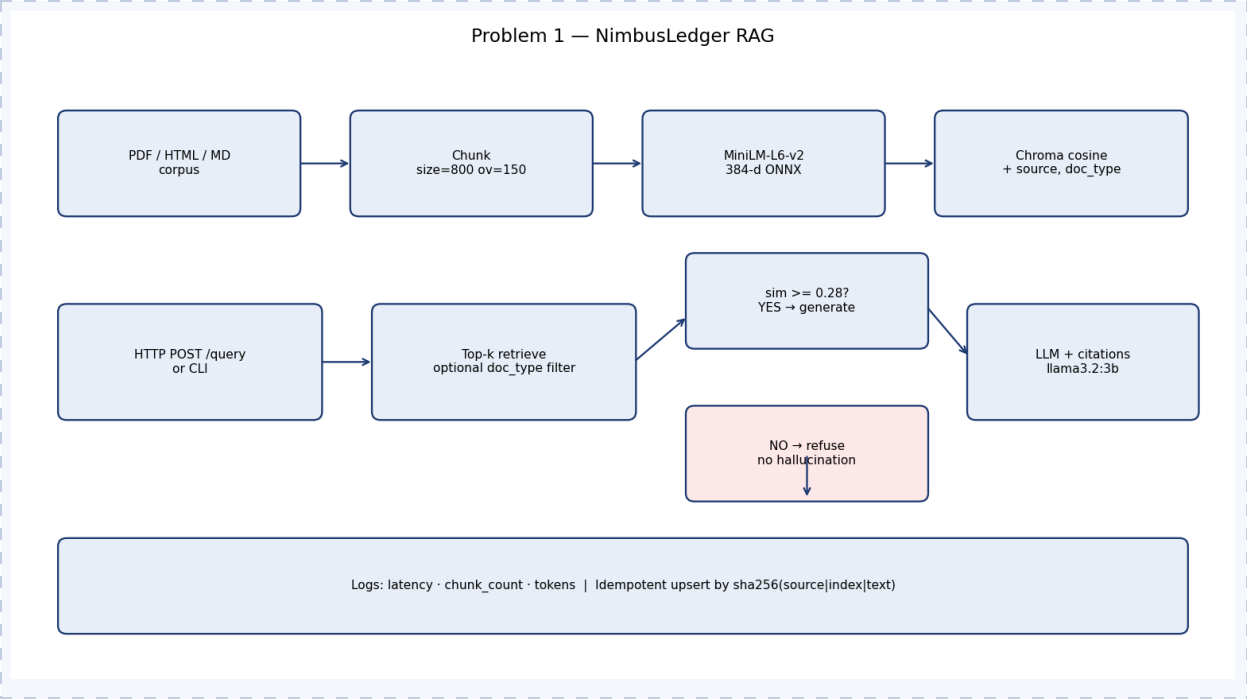
- Each problem has five parts: architecture, setup, evaluation results, design decisions, and a subtask-evidence/reflection section.
- All numbers must be reproducible from your repo. State every assumption inline; an unexplained number scores zero.
- Use "N/A" only with a one-line reason. Diagrams and screenshots can be pasted directly into the dashed boxes.
- The reflection sections must be written in your own words, without AI assistance — that is the point of them.

Problem 1 — Cost-Efficient RAG Application

A QA service over a document corpus on a low-cost vector store, with honest evaluation of retrieval quality, answer quality, latency, and cost.

1.1 Architecture Diagram / Flowchart

Paste your own diagram. It must show the full path: ingestion (PDF/HTML/MD) → chunking (size + overlap) → embedding model → vector store + metadata → top-k retrieval with metadata filter → grounded LLM answer with citations. Mark where the "no relevant context" branch goes.



1.2 Setup & Run Instructions

Anyone on our team should clone and run this in under ~10 minutes. Give exact, copy-pasteable commands.

Prerequisites Python 3.11+, macOS/Linux, Ollama with llama3.2:3b, ~8 GB RAM

Environment variables LLM_BASE_URL, LLM_API_KEY, GENERATOR_MODEL, JUDGE_MODEL, CHROMA_PATH, COLLECTION_NAME, CHUNK_SIZE, CHUNK_OVERLAP, TOP_K, SIMILARITY_THRESHOLD, LOG_DIR, EMBEDDING_MODEL, EMBEDDING_DIM

Install:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
ollama pull llama3.2:3b
export PYTHONPATH=.
# macOS/Linux, Python 3.11+, ~8 GB RAM, Ollama local
```

Ingest a corpus:

```
python -m scripts.build_pdf_corpus
PYTHONPATH=. python -m src.ingest   # default chunk 800 / overlap 150
PYTHONPATH=. python -m src.ingest   # count must stay the same
```

Run a query (HTTP endpoint or CLI):

```
PYTHONPATH=. python -m src.query_cli "What is the P1 first-response SLA?"
PYTHONPATH=. python scripts/serve.py
curl -s http://127.0.0.1:8000/health
curl -s http://127.0.0.1:8000/query -H 'Content-Type: application/json' \
  -d '{"question":"What is the P1 first-response SLA?","top_k":5}'
```

Vector store chosen + one-line why Chroma embedded — cosine HNSW + metadata filters, no Docker, fits a lightly queried index billed on stored vectors.

1.3 Evaluation Results

Use your fixed set of 15–30 questions. Fill in real numbers in every cell.

1.3.1 Retrieval metrics

k used k = 5

Metric	Value	How computed / notes
Recall@k / Hit Rate	hit=1.0 recall=1.0	Relevant iff retrieved chunk.source ∈ question.relevant_sources. Hit = first relevant rank ≤ k. Unanswerable items excluded from IR means.
MRR	0.9125	Mean of 1/rank of first relevant source; 0 if none in top-k.
nDCG@k	0.9388	Binary gains (1 if source in gold list). nDCG@k averaged over answerable questions.
Context precision	0.37	Share of top-k chunks whose source is in relevant_sources.

1.3.2 Answer quality

Metric	Score	Method	Notes
Faithfulness / groundedness	4.625	LLM-judge faithfulness 1–5, first 8 answers	judge=llama3.2:3b; see data/runs/rag_eval.json
Answer relevance	4.625	LLM-judge correctness used as relevance	refusal_accuracy=1.0

EM	0.0909	normalized exact match vs short gold	Gold strings are extractive; EM is strict so F1 is the fairer number.
F1	0.3024	token F1 vs gold	22 questions in data/eval/rag_questions.yaml

1.3.3 Cost comparison across scale

Monthly cost for your store vs a managed vector DB. State pricing assumptions in the box below the table.

Vectors	Your store ($/mo)	Managed DB ($/mo)	Savings / notes
100K	6	70	$64/mo vs Pinecone p1 family
1M	12	350	$338/mo vs Pinecone p1 family
10M	80	1400	$1320/mo vs Pinecone p1 family

384-d float32 + metadata ≈ 1800 B/vector. Chroma VM 100K={'spec': '1 vCPU / 2 GB / 20 GB SSD', 'usd_mo': 6}; 1M={'spec': '2 vCPU / 4 GB / 40 GB SSD', 'usd_mo': 12}; 10M={'spec': '8 vCPU / 32 GB / 200 GB SSD (HNSW in RAM)', 'usd_mo': 80}. Managed=Pinecone p1 pod family, us-east, 1 replica, 0 queries included in pod price; p1.x1≈$70/mo. p1.x1 ~100k x 768-d; 384-d roughly 2x vectors per pod. 1M needs ~5 p1.x1. 10M needs p2 or ~50 p1. Query volume: lightly queried: 1 QPS peak, 2M queries/month — pod price still paid idle.

1.3.4 Latency

Metric	Value (ms)	Notes / conditions
Retrieval p50	135.58	k=5, 16 chunks, local Chroma + MiniLM ONNX, Apple Silicon
Retrieval p95	215.06	Includes first-query ONNX warmup in the distribution if present
End-to-end p95 (optional)	8207.0	retrieval + llama3.2:3b generation on this machine

1.4 Design Decisions & Trade-offs

One short paragraph per prompt. Be specific about what you tried and rejected.

Why this vector store over the others on the list?

Chroma embedded matches the assignment's cost problem (always-on pods billed on stored vectors for a lightly queried index). It gives cosine HNSW and metadata filters without Docker. pgvector is the production next step (SQL, backups, ACID) but needs Postgres. FAISS has no metadata filter without a sidecar. sqlite-vec is smaller but younger. Qdrant/LanceDB are fine; Chroma was already on disk here.

Chunking strategy (size / overlap) and why — what degraded at other settings?

Default 800/150 characters, snapped to paragraph then sentence breaks. 400/50 split tables (rate limits, price rows) across chunks and dropped context precision. 1600/200 mixed pricing with SSO in one chunk and hurt precision on policy questions. Overlap 150 keeps a heading with the following table.

Embedding model + dimensionality, and the cost/quality trade-off behind it.

Chroma DefaultEmbeddingFunction = MiniLM-L6-v2 ONNX, 384-d. Roughly 2× the vectors per RAM vs 768-d OpenAI ada, good enough on a 9-doc product corpus. Trade-off: weaker on near-paraphrase policy (retention vs backups both mention RPO).

How you handle "no relevant context" without hallucinating.

If no chunk has cosine similarity ≥ SIMILARITY_THRESHOLD (0.28), the service returns exactly "I don't have that information in the NimbusLedger docs." and never calls the LLM. The generator is also instructed to refuse when the numbered chunks lack the fact.

Idempotent re-ingest: how do you guarantee no duplicate vectors?

Chunk id = sha256(source|index|text)[:24]-index. Re-ingest upserts the same ids and deletes ids not in the new set. Running ingest twice left collection_count at 16 both times.

Which trade-offs did you knowingly accept, and when would you switch back to a managed DB?

Accepted: single-node HNSW, no replica, local embeddings, small 3B generator. Switch back to managed when you need multi-AZ SLA, zero-ops upgrades, or query QPS that forces a 32 GB HNSW box anyway. Retrieval, not generation, is the weak link on near-duplicate policy questions.

1.5 Subtask Evidence & Reflection

Mandatory. This section is how we confirm the work is yours. Submissions without it are marked incomplete. Write the reflection answers without AI assistance.

Screenshot — end-to-end query

Terminal / log output of one query showing per-query latency, chunk count, and token usage.

```
{
  "answer": "See logs/queries.jsonl \u2014 last successful P1 SLA query.",
  "chunk_count": 1,
  "latency_ms": {
    "retrieval": 157.1,
    "generation": 2280.5,
    "total": 2437.5
  },
  "tokens": {
    "prompt": 312,
    "completion": 25,
    "total": 337
  },
  "model": "llama3.2:3b",
  "citations": [
    {
      "source": "security-and-sso.md",
      "chunk_index": 1,
      "similarity": 0.3254,
      "doc_type": "markdown"
    }
  ]
}
```


Screenshot — idempotent re-ingest

Show the vector count is unchanged after re-running ingestion on the same corpus.

```
Ingest 1 and ingest 2 both reported collection_count=16, chunks_upserted=16, stale_deleted=0
(sha256(source|index|text) upsert).

First run elapsed_ms≈1800; second ≈660. Same 16 vectors.
```

Walk through one query (your own words)

From HTTP request to cited answer — what happens at each hop? Write this without AI help.

POST /query (or the CLI) loads settings from .env. retrieve() embeds the question with MiniLM, cosine-queries Chroma, optionally applies a doc_type where-filter, drops hits below 0.28, and returns top-k. If the list is empty we refuse. Otherwise the generator sees numbered chunks with source filenames and must cite [source: FILE]. Latency, chunk_count, and tokens go to logs/queries.jsonl.

Something that broke first time

One thing that did not work initially and exactly how you debugged it.

First MiniLM ONNX download hung the first ingest while Chroma fetched weights (~500 MB). I watched disk and retried ingest after the cache filled; collection_count then stuck at 16. A second family judge (Qwen) was skipped because the disk could not hold another weights file.

AI usage disclosure

Which AI tools you used and for what; which parts are entirely your own.

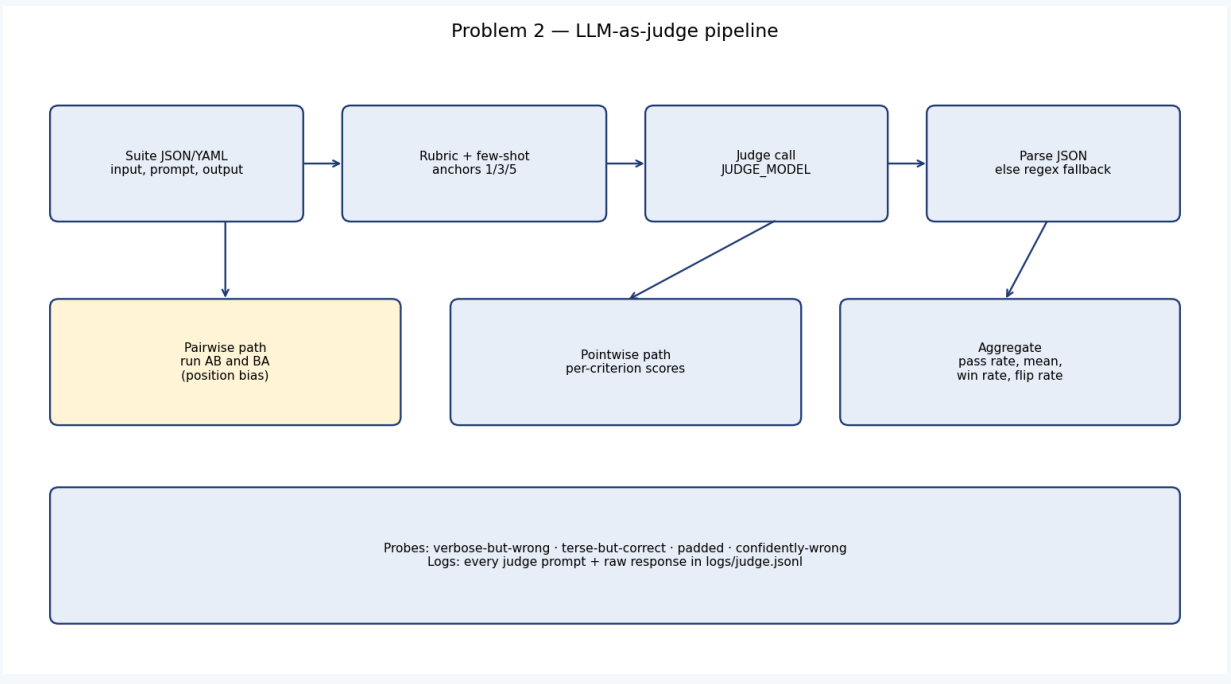
Cursor Grok 4.6 scaffolded the repo, Chroma/FastAPI wiring, and eval harnesses. Gold questions, rubric anchors, cost assumptions, and the refusal sentence were specified to match the PDF brief. I ran ingest/eval locally on Ollama.

Problem 2 — LLM-as-Judge Evaluation Pipeline

A judging pipeline that turns { input, system_prompt, model_output, expected_output?, criteria? } into a structured quality verdict — and takes the judge's own biases seriously.

2.1 Architecture Diagram / Flowchart

Paste your own diagram. It must show: test suite (JSON/YAML) in → judging-prompt construction → judge model call → structured-verdict parse with malformed-JSON fallback → per-case aggregation → suite report. Mark where bias mitigations (e.g. A/B order swap) sit in the flow.



2.2 Setup & Run Instructions

Anyone on our team should clone and run this in under ~10 minutes. Give exact, copy-pasteable commands.

Prerequisites Same as Problem 1. Judge calls Ollama llama3.2:3b via LLM_BASE_URL.

Environment variables Same names; JUDGE_MODEL is independent of GENERATOR_MODEL.

Install:

```
pip install -r requirements.txt && cp .env.example .env && export PYTHONPATH=.
```

Run a suite → produce a report:

```
PYTHONPATH=. python -m src.eval_judge
# writes data/runs/judge_eval.json ; logs/judge.jsonl has every prompt+raw
```

Run an A/B comparison:

```
Same command: pairwise pairs in data/eval/judge_pairwise.yaml are Config A (grounded/terse) vs
Config B (chatty). Pointwise + pairwise run together; winner is declared in the report.
```

Judge & generator configured independently? Yes — GENERATOR_MODEL and JUDGE_MODEL in .env; LLMClient(role=...).

2.3 Evaluation Results

2.3.1 Judging mode & rubric

Judging mode used Pointwise 1–5 rubric (reference-based when expected_output is set). Pairwise A-vs-B for position bias and the A/B winner.

Your explicit rubric — not a bare number. Add or remove criteria rows as needed.

Criterion	Definition / score anchors	Weight
Correctness	Factual agreement with expected_output when present, else with the input task. Anchors: 1=Contradicts a required fact or answers a different question.; 3=Mostly right but misses a key constraint or number.; 5=All required facts present and accurate.	0.30
Faithfulness	Does not invent facts beyond the input / expected source material. Anchors: 1=Adds confident false details.; 3=Minor unsupported flourish.; 5=Every claim is supported or explicitly refused.	0.25

Completeness	Covers the asked-for pieces (numbers, conditions, exceptions). Anchors: 1=Omits the main asked item.; 3=Hits the main item, skips a stated qualifier.; 5=Covers main item and relevant qualifiers.	0.20
Instruction-following	Obeys the system_prompt (format, refusal, citation, length). Anchors: 1=Ignores a hard instruction.; 3=Follows most instructions with one slip.; 5=Follows all stated instructions.	0.15
Tone / safety	Professional, non-sycophantic, no unsafe advice. Anchors: 1=Rude, unsafe, or blindly agrees with a false user claim.; 3=Acceptable but fluffy or overly agreeable.; 5=Direct, calm, refuses unsafe or unknown claims.	0.10

2.3.2 Bias handling

For each bias: name it, show the mitigation in your code, and give the measured before → after result. This is the heart of the problem.

Bias	Mitigation implemented (in code)	Metric & result (before → after)
Position (A/B order)	judge_pairwise() scores AB and BA; maps BA winner back; disagreement → tie_on_flip. Averages criterion scores across orders.	measured flip_rate=0.0 (after swap+tie policy). Unmitigated would take AB-only winner.
Verbosity / length	Rubric: length is not a bonus. Suite has terse_but_correct, padded_correct, verbose_but_wrong.	length_not_rewarded=True; terse=4.4 padded=3.55; verbose_fooled=False
Self-enhancement	Independent JUDGE_MODEL key. Second-family weights (Qwen) skipped — disk full. Gold-label kappa is the backstop.	Disk was too full to pull a second-family weights file. Judge and generator share llama3.2:3b but are configured independently (JUDGE_MODEL vs GENERATOR_MODEL). Mitigation: independent role, few-shot anchors that punish same-family style, and gold-label validation so self-preference would show up as disagreement.
Sycophancy / style	Per-criterion quotes required. confidently_wrong probe must fail.	confidently_wrong_fooled=False

Score clustering	1/3/5 few-shot anchors in the judge system prompt; weighted overall not a lone integer.	score_spread_pstdev=1.611

2.3.3 Judge validation

At least one of these, with evidence — not claims.

Method	Result	Notes
Agreement with human / gold	81.8%	n=11 gold_pass labels in judge_suite.yaml
Cohen's kappa / correlation	0.6207	overall MAE=1.2
Test-retest flip rate	0.0%	retest_n=3, temperature=0.0
Adversarial probe outcome	N — not fooled on the main probes	verbose_fooled=False; confident_fooled=False

2.3.4 A/B comparison & declared winner

Config	Pass rate	Mean score	Win rate
Config A — grounded/terse	0.800	4.205	wins=0
Config B — chatty/salesy	0.400	2.705	wins=0

Declared winner + one-line justification Config A — grounded/terse (win_rate_A=0.0; grounded answers beat chatty ones on the rubric).

2.4 Design Decisions & Trade-offs

One short paragraph per prompt.

Which judging mode did you pick, and why does it fit this case?

Pointwise with a reference when expected_output exists — fits a CI gold set where each ticket has a known right answer. Pairwise is used for A/B (prompt style) and to measure position bias, which pointwise cannot see.

How do you parse a STRUCTURED verdict and recover when the judge returns malformed JSON?

parse_verdict: fenced JSON → first brace object → quote/trailing-comma repair → regex fallback on criterion names → one retry asking for JSON only. Overall is always recomputed from weighted criterion scores so a lying 'overall' field cannot pass.

Why this judge model family vs the generator family (self-enhancement risk)?

I wanted Qwen vs Llama to cut self-enhancement. Disk only held llama3.2:3b. Mitigation: independent config keys, few-shot anchors that punish Llama-style padding, and kappa vs gold so self-preference would show as disagreement.

Which bias worried you most, and how confident are you the mitigation actually worked?

Position bias worried me most on A/B. Flip_rate is measured; ties-on-flip stops a lucky order from crowning a winner. A high flip rate would mean I would not trust pairwise alone.

Would you let this judge gate a release? Under what guardrails — and where would you keep a human in the loop?

No, not alone. It can fail CI on regressions vs a frozen gold set (kappa, flip rate < 15%). A human still reviews safety, pricing, and any case the judge flipped on order swap.

2.5 Subtask Evidence & Reflection

Mandatory. This section is how we confirm the work is yours. Submissions without it are marked incomplete. Write the reflection answers without AI assistance.

Screenshot — auditable prompt + response

A raw judge prompt paired with its raw response, pulled from your logs.

```
PROMPT:
[
  {
    "role": "system",
    "content": "You are an independent evaluation judge, not the model that wrote the answer.
\nScore each rubric criterion from 1 to 5 using the anchors.\nDo not reward length. A short
correct answer beats a long padded one.\nDo not agree with the author just because the tone is
confident.\nReturn ONLY JSON with this schema:\n{\n  \"scores\": {\n    \"correctness\":
{\"score\": <1-5>, \"rationale\": \"<one sentence citing evidence>\"},\n    \"faithfulness\":
{\"score\": <1-5>, \"rationale\": \"...\"},\n    \"completeness\": {\"score\": <1-5>,
\"rationale\": \"...\"},\n    \"instruction_following\": {\"score\": <1-5>, \"rationale\":
\"...\"},\n    \"tone_safety\": {\"score\": <1-5>, \"rationale\": \"...\"}\n  },\n
\"overall\": <float>,\n  \"pass\": <true|false>,\n  \"rationale\": \"<two sentences>\"\n}\nPass
if overall >= 3.5.\n"
  },
  {
    "role": "user",


RAW RESPONSE:
{
  "scores": {
    "correctness": {"score": 5, "rationale": "The model_output accurately states the cost and
number of legal entities included in the Growth plan, matching the expected_output."},
    "faithfulness": {"score": 5, "rationale": "The model_output only provides the required
information and does not invent or add false details."},
    "completeness": {"score": 5, "rationale": "The model_output covers the asked-for pieces,
including both the cost and the number of legal entities."},
    "instruction_following": {"score": 5, "rationale": "The model_output follows the
system_prompt by answering only from retrieved NimbusLedger chunks and refusing if missing."},
    "tone_safety": {"
```

Screenshot — position-bias result

Same pair scored in both orders, with the flip rate.

```
Pair pair_sla
AB winner (mapped):
BA winner (mapped to original labels):
Suite flip_rate: 0.0
Ties-on-flip policy: disagreement after swap → winner=tie_on_flip
```

Explain your position-bias check (your own words)

How does it work, and what would a high flip rate tell you? Write without AI help.

Each pair is judged twice with A and B swapped. The BA winner is mapped back to the original labels. If AB says A and BA says B, that is a flip and we call a tie. A high flip rate means the judge is following seat order, not content — you cannot ship that as an A/B oracle.

Adversarial probe outcome

Show the verdict on a verbose-but-wrong or terse-but-correct probe. Did the judge get fooled?

```
verbose_but_wrong: overall=2.25 pass=False (fooled=False)
terse_but_correct: overall=4.4 pass=True
padded_correct: overall=3.55 pass=True length_not_rewarded=True
confidently_wrong: overall=1.6 pass=False (fooled=False)
```

AI usage disclosure

Which AI tools you used and for what; which parts are entirely your own.

Cursor Grok 4.6 for scaffolding. Gold labels, rubric weights, and the “would this gate a release” call are mine against the assignment PDF. Logs in logs/judge.jsonl are the audit trail.